

EE3703 : Assignment 6:

Speeding up with Cython

RESULTS:

1. Function: $f(x) = x^2$:

Actual Value = 0.333333...

	n = 1000		n = 100000	
Method:	Time	Value	Time	Value
Python	756 μ s $\pm$ 229 μ s	0.33333349999999984	66.9 ms $\pm$ 1.42 ms	0.33333333335000015
Numpy	783 μ s $\pm$ 222 μ s	0.33333335	66.8 ms $\pm$ 7.13 ms	0.333333333335
Cython	537 μ s $\pm$ 135 μ s	0.33333349999999984	40.9 ms $\pm$ 10.9 ms	0.333333333335000015

2. $f(x) = \sin(x)$

Actual Value = 2

	n = 1000		n = 100000	
Method:	Time	Value	Time	Value
Python	871 μ s $\pm$ 234 μ s	1.999998355065662	79.1 ms $\pm$ 19.5 ms	1.9999999998355185
Numpy	661 μ s $\pm$ 13.3 μ s	1.9999983550656628	65.5 ms $\pm$ 1.17 ms	1.9999999998355067
Cython	303 μ s $\pm$ 8.63 μ s	1.999998355065662	30.8 ms $\pm$ 989 μ s	1.9999999998355185

3. $f(x) = e^x$,

Actual Value = $e - 1 = 1.718281828459045....$

	n = 1000		n = 100000	
Method:	Time	Value	Time	Value

Python	718 $\mu\text{s} \pm 5.13 \mu\text{s}$	1.71828197164 9196	88 ms ± 24.2 ms	1.71828182847 33672
Numpy	815 $\mu\text{s} \pm 240 \mu\text{s}$	1.71828197164 9195	69 ms ± 4.41 ms	1.71828182847 3364
Cython	371 $\mu\text{s} \pm 68.8 \mu\text{s}$	1.71828197164 9196	32.7 ms ± 1.13 ms	1.71828182847 33672

4. $f(x) = 1/x$

Actual Value = $\ln(2) = 0.693147....$

n	n = 1000		n = 100000	
Method:	Time	Value	Time	Value
Python	712 $\mu\text{s} \pm 216 \mu\text{s}$	0.69314724305 99359	58.2 ms ± 674 μs	0.69314718056 61945
Numpy	606 $\mu\text{s} \pm 9.53$	0.69314724305 99375	80.3 ms ± 22.2 ms	0.69314718056 61953
Cython	254 $\mu\text{s} \pm 70.4 \mu\text{s}$	0.69314724305 99359	21.1 ms ± 875 μs	0.69314718056 61945

Final Answer for 5):

Actual Value of the integral is 333.3333..., When $n = 10^7$ (10 million). The execution times are **7.89 s $\pm$ 905 ms (Numpy), 7.42 s $\pm$ 531 ms (Python), 3.12 s $\pm$ 421 ms (Cython)**.

Clearly the time taken by Cython is much smaller, making it always 2x faster than Python and Numpy.

Observations:

As n increases, The accuracy increases or the value becomes closer to the actual value of the integral and time for computation increases. The time taken by Cython is much less than Python and Numpy. The values computed by Python and Cython are very much alike and accurate and when comparing it to Numpy, the values given by Py and Cy are more or less the same. Hence performance with Cython is much better.

Code Implementation:

For the **numpy implementation**, I've defined a function that creates a list of the $n - 1$ points between a and b and defined a function called `py_trapz` that computes the area of trapeziums after dividing the x axis and the equivalent `cy_trapz` that executes the same using Cython.

